

CLINICAL AI CONTROL PLANE

Model Provider and Redaction Research

CAG-05 · Production Seam Feasibility

Prepared by Guillermo (Bill) Cuevas · 2026-08-21

PHI PROTECTION · FULL AUDIT TRAIL · HIPAA ALIGNED

Clinical AI Control Plane: Model Provider and PHI Redaction Research

Feasibility of the production seams · provider options · de-identification options · local dev

SECTION 00

Headline finding (read this first)

The governance plane is provider-proof by construction. Every candidate model backend and every candidate de-identification service plugs in behind an existing interface (`ModelProvider` , `RedactionService`) without touching metering, policy, persistence, or audit. Vendor selection is therefore a deployment decision with a contained blast radius, not an architecture decision. The two real constraints found are: the `ModelResult` contract (provider, output, tokens) must stay uniform across backends so token accounting and cost reporting do not fork, and redaction quality, not redaction placement, is the open engineering problem for free-text clinical notes.

SECTION 01

How the provider seam constrains the pipeline

- **The seam is called after redaction and inside metering.** Any backend receives only redacted text, and its token usage is recorded to the per-tenant meter. A provider integration must report token counts; a backend that cannot report usage breaks cost observability and needs a token-estimation shim.
- **The result contract is the invariant:** `ModelResult(provider, output, tokensUsed)` . Provenance stores the provider string per suggestion, so responses must be attributable to a specific backend and model version for the audit trail to stay meaningful. Recommendation: extend `provider` to carry model id and version when the remote provider is wired.
- **Synchronous today, async-ready:** the model call sits on the request path. Latency budgets for real backends (hundreds of ms to seconds) may require the call to become asynchronous; ADR 0001 anticipated this, and the seam means the change does not disturb governance.
- **Routing and fallback compose in front of the seam:** a routing provider that wraps several concrete providers satisfies the same interface, which is where model tiering (small model for drafts, large model for clinical text) belongs.

SECTION 02

Model backend options

OPTION	WHAT IT IS	STRENGTHS	CONSIDERATIONS	FIT
AWS Bedrock	Managed multi-model API (Claude and others) inside the AWS account	HIPAA-eligible service; traffic stays in the AWS boundary; IAM-native auth; fits the existing EKS/Terraform footprint	Model availability and pricing vary by region; per-model invocation quotas need capacity planning	**Best fit for the current AWS deployment target**
Anthropic Claude API (direct)	First-party hosted API	Latest models first; strong clinical reasoning; token usage returned per call, mapping cleanly to `ModelResult`	Requires a business associate agreement for PHI-adjacent use; traffic leaves the AWS boundary	Strong fit where the newest model capability matters most
Self-hosted open model (vLLM on EKS)	Open-weights model served in-cluster	Data never leaves the cluster; no per-token vendor cost; full version pinning	GPU node groups, serving expertise, and evaluation burden shift in-house; capability gap for clinical text	Fallback or data-residency option, not the first integration
Mock provider (current default)	Deterministic, specialty-shaped output in-process	Zero cost, zero latency, deterministic tests, no cloud dependency	Not a product capability; exists to prove the plane	Remains the default for local dev and CI permanently

Recommendation. Wire `RemoteModelProvider` against Bedrock first: it keeps PHI-adjacent traffic inside the existing AWS trust and compliance boundary and requires no new credential surface beyond IAM. Keep the direct Anthropic API as a second concrete provider behind the same seam, selected per environment, and revisit routing once two real providers exist.

SECTION 03

PHI de-identification options

The current implementation is pattern-based (SSN, phone, email styles of direct identifiers) and documented as illustrative. ADR 0004 fixes the placement (gateway boundary, always on) and leaves the detection engine swappable. Candidates:

OPTION	WHAT IT IS	STRENGTHS	CONSIDERATIONS	FIT
AWS Comprehend Medical	Managed clinical NLP with a PHI detection API	Purpose-built for clinical text; HIPAA-eligible; managed, no model to maintain; entity types map to HIPAA identifiers	Per-request cost on every gateway call; adds a network hop to the hot path; English-centric	**Best fit for production on the AWS target**
Microsoft Presidio	Open-source PII/PHI detection and anonymization framework	Runs in-cluster, no per-call cost, extensible recognizers, deterministic and testable	Clinical recall depends on the NLP models configured; tuning and evaluation are owned in-house	Strong fit for cost control and data residency; good staging step
Commercial clinical NLP (e.g. John Snow Labs)	Licensed clinical de-identification pipelines	State-of-the-art published accuracy on clinical de-identification benchmarks	License cost; heavier runtime footprint; procurement	Evaluate if measured recall of the first choice is insufficient
Pattern-based (current)	Regex rules for direct identifiers	Zero dependency, fast, fully deterministic for tests	Not sufficient for free-text clinical notes; no recall guarantees	Remains the test/dev engine and the documented seam marker

Recommendation. Comprehend Medical behind `RedactionService` for production, with the pattern engine retained as the deterministic test double. Define a measured acceptance bar before switching: a labeled evaluation set of representative notes, recall on HIPAA identifier categories, and an explicit over-redaction budget, since over-redaction degrades model output quality (the tradeoff ADR 0004 records).

SECTION 04

Hallucination posture: provenance and review as mitigations

- **Grounding references are the anchor.** Every suggestion carries typed references to the source records (encounter, note) it was generated from. A suggestion that cannot cite grounding is reviewable on that basis alone, and downstream validators can check output claims against the referenced sources.
- **The review policy is the risk valve.** Where hallucination cost is high (clinical, billing), `REVIEW_REQUIRED_ASSISTANTS` routes exactly those flows to a human without slowing the rest of the fleet.
- **The audit trail makes model behavior measurable.** Because every suggestion stores provider, output, tokens, and decision outcome, rejection rates per assistant type and per provider are queryable, which is the raw material for an evaluation loop (flagging providers or prompts whose rejection rate drifts).
- **Next step, not yet built:** an automated validation stage between the provider call and persistence (schema checks on structured outputs, grounding-consistency checks) behind its own seam, so validation evolves without touching orchestration.

SECTION 05

Local development workflow

- The mock provider keeps the entire governed flow runnable and testable with **no cloud account, no API key, and no model cost**: Docker Compose supplies Postgres, Redis, and Kafka; the dev profile mints local tokens.
- The integration test proves the plane end to end (HTTP in, audit trail out) with Testcontainers; it will not change when a real provider is wired, because it exercises governance, not the vendor.
- When the remote provider lands, contract tests against recorded provider responses keep CI deterministic; live-provider smoke tests run in a separate, credentialed pipeline stage, never on the default build.

SECTION 06

Open items to close

- [ ] Confirm business associate agreement coverage for the chosen backend (Bedrock service terms, or a direct Anthropic BAA).
- [ ] Extend `ModelResult.provider` to carry model id and version for provenance before the first real integration.
- [ ] Build the labeled de-identification evaluation set and set the recall and over-redaction acceptance bar.
- [ ] Benchmark request-path latency with a real provider; decide whether the model call moves off the synchronous path.
- [ ] Design the validation stage (schema and grounding-consistency checks) as its own seam.